UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Sistemas Operativos

Diseño de Syscalls

Práctica 2

Presenta:
Rojo Peña Manuel Ianluck

Profesor:
Javier León Cotonieto

Ayudante laboratorio:
Adrián Martínez Manzo

Grupo: 7004, 2026-2

Fecha de entrega:
5 de marzo, 2026

Introducción

Las llamadas al sistema (syscalls) representan el mecanismo mediante el cual un programa de usuario puede solicitar al sistema operativo que ejecute tareas u operaciones privilegiadas en beneficio del usuario y el sistema.

Este mecanismo establece una barrera de protección y seguridad entre ambos: el **Modo Usuario** (*no privilegiado*), donde corre la lógica de aplicación, y el **Modo Kernel** (*privilegiado*), donde residen los controladores de hardware y los servicios del sistema. Para esta práctica, la restricción es que: el código de usuario no puede incluir bibliotecas de hardware ni utilizar apuntadores directos a registros físicos. Si el usuario desea realizar cualquier acción (presionar un botón para activar/desactivar un LED), se debe realizar una llamada al sistema, cediendo el control al procesador para que el Kernel verifique y ejecute la acción en su nombre.

Planteamiento del Problema

El reto consiste en que, permitir que una aplicación de usuario acceda directamente a la memoria de periféricos es una vulnerabilidad crítica de seguridad en cualquier sistema operativo moderno. Un acceso directo sin validación puede:

- Corromper registros de hardware compartidos entre múltiples procesos.
- Habilitar ataques de escalada de privilegios.
- Generar fallos de hardware irreversibles al manipular pines reservados del sistema.

La solución es implementar una capa de abstracción mediante una **ABI** (*Application Binary Interface*) definida por el estándar ARM AAPCS, usando la instrucción `SVC` como única puerta de entrada al modo privilegiado. El programa de usuario únicamente conoce identificadores numéricos de operaciones; el kernel decide si ejecutarlas o rechazarlas.

Conceptos Necesarios

ABI (*Application Binary Interface*)

Una **ABI** define las reglas de bajo nivel que deben seguir los programas para comunicarse entre sí a nivel de código máquina. A diferencia de las *API*, la **ABI** especifica exactamente cómo los datos son pasados entre funciones: qué registros del procesador se usan para argumentos, cuál lleva el valor de retorno, cómo se estructura el *stack*, y cómo se preservan los registros entre llamadas.

Para esta práctica implementamos una **ABI** personalizada sobre el estándar **ARM AAPCS** (*ARM Architecture Procedure Call Standard*) para la comunicación entre el `modo usuario` y el `modo kernel`. Las reglas que aplicamos son:

Elemento ABI	**Registro**	**Descripción**
ID de Syscall	r7	Identifica qué operación solicita el usuario
Argumento 1	r0	Primer parámetro (ej. número de pin GPIO)
Argumento 2	r1	Segundo parámetro (ej. valor del pin/btn)
Valor de retorno	r0	El kernel deposita el resultado en r0
Instrucción de entrada	svc #0	Genera la excepción de software al kernel

Supervisor Call (*SVC*)

Esta instrucción de excepción **SVC** (antes llamada **SWI** *Software Interrupt*) es especial del conjunto **ARM** que genera una excepción de software de forma intencional y controlada. Es el mecanismo oficial para que el código no privilegiado transfiera el control al kernel de forma segura.

Cuando se ejecuta `svc #0`, ocurre lo siguiente de forma automática en la pico:

- El procesador termina la instrucción actual y entra en modo **Handler** (privilegiado).
- Guarda automáticamente un **Stack Frame** con 8 registros en el **Main Stack**: $r0$, $r1$, $r2$, $r3$, $r12$, LR, PC y $xPSR$.
- "Salta" al vector de excepciones definido en la tabla de vectores, donde apunta el manejador **SVC** (`wrapper_svc`).
- El kernel toma control total del procesador.

La clave de seguridad está en que el usuario no puede elegir a dónde salta el procesador: ese destino está fijo en la tabla de vectores, que solo el kernel puede modificar. Esto nos lleva a la siguiente definición.

Main Stack Pointer (*MSP*)

Es el registro del procesador que apunta al tope del stack principal (su traducción literal pues). Se usa en modo **Handler** (*kernel*), mientras que los procesos de usuario pueden usar el **PSP** (*Process Stack Pointer*).

Cuando ocurre la excepción SVC, el ARM guarda el Stack Frame en el stack activo del momento (en este caso el MSP) y el `wrapper_svc` lee el valor del MSP con la instrucción:

```
mrs r0, MSP @ Lee el Main Stack Pointer y lo coloca en r0
```

Este apuntador es sumamete necesario porque apunta exactamente al inicio del Stack Frame guardado por el hardware, el kernel en C recibe ese apuntador como `svc_args[]` y puede así leer los registros $r0$-$r3$ del usuario (que son los argumentos de la syscall) y escribir de vuelta en $r0$ para retornar un valor.

Esto debido a que, cuando ARM guarda el Stack Frame, lo hace en este orden exacto en memoria:

```
[SP + 0] = r0 ← svc_args[0]   (primer argumento / valor de retorno)
[SP + 4] = r1 ← svc_args[1]   (segundo argumento)
[SP + 8] = r2 ← svc_args[2]
[SP + 12]= r3 ← svc_args[3]
[SP + 16]= r12
[SP + 20]= LR
[SP + 24]= PC     (dirección de retorno)
[SP + 28]= xPSR
```

Escribir en `svc_args[0]` es literalmente escribir en r0 del usuario, por eso el kernel puede "retornar" un valor al usuario sin ninguna instrucción especial.

Arquitectura del Sistema

Recordemos que tenemos dos modos:

Modo USUARIO (No Privilegiada)

- `syscalls.S`: Biblioteca **ABI** QUE oculta el ensamblador tras funciones C.
- `user_app.c`: La lógica de aplicación, solo usa syscalls nunca hardware directo.

Modo Kernel (Privilegiada)

- `main.c`: Configura la tabla de vectores y lanza la aplicación de usuario.
- `svc_handler.S`: **Wrapper ASM** extrae el MSP y al $r7$ y llama a `kernel_service()`.
- `kernel_service.c`: El dispatcher, valida el ID y decide qué driver invocar.
- `kernel_drivers.c`: Único archivo con acceso a `0xD0000000` (`SIO/GPIO RP2040`).

Flujo completo de una syscall

Paso 1. El Usuario llama a una función C (`user_app.c`).

El código de la aplicación de usuario invoca una función de la biblioteca de syscalls como si fuera una función C ordinaria. No tiene idea de que internamente hay ensamblador ni registros ARM.

```
// En user_app.c sys_gpio_set(LED_PIN, LED_STATE);
```

Paso 2. El Ensamblador prepara la **ABI** (`syscalls.S`)

La función `sys_gpio_set` no es C, es una función escrita en ensamblador ARM Thumb, su único trabajo es cargar el ID de syscall en $r7$ y luego ejecutar `svc #0`.

```
@ syscalls.S sys_gpio_set:   @ r0 ya tiene el pin (13) colocado
por el compilador C  @ r1 ya tiene el valor (1) colocado de
nuevo por el compilador C
 mov r12, r7 @ Preservamos el r7 del usuario en r12 (caller-saved)
movs r7, #1 @ Carga el ID de syscall:  SYS_GPIO_SET = 1  svc
#0 @ EXCEPCION DE SOFTWARE - transfiere control al kernel  mov
r7, r12 @ Restaura r7 al valor original del usuario  bx lr @
Regresamos al código usuario
```

La instrucción `mov r12, r7` es necesaria porque $r7$ podría tener un valor importante del contexto del usuario. Al guardarlo en $r12$ y restaurarlo después del `svc`, la función respeta la convención **ABI** sin tocar el stack del usuario.

Paso 3. El Hardware ARM guarda el Stack Frame automáticamente

Al ejecutar `svc #0`:

- Detecta la excepción SVC.
- Guarda en el stacklos 8 registros del estado actual del usuario: r0, r1, r2, r3, r12, LR, PC, xPSR.
- Cambia al modo Handler (modo privilegiado).
- Salta al vector de excepción SVC, donde está registrado `wrapper_svc`.

Paso 4. El Wrapper ASM extrae los argumentos (`svc_handler.S`)

El manejador de excepción en ensamblador recibe el control del procesador, su función es extraer los dos datos que `kernel_service()` necesita y hacer el puente hacia C:

```
@ syscalls.S wrapper_svc:   mrs r0, MSP @ Lee el Main Stack
Pointer  @ r0 ahora apunta al inicio del Stack Frame  @ Este
sera el argumento svc_args[] en C
 mov r1, r7 @ r7 contiene el ID de syscall que pusimos en el
Paso 2  @ Este sera el argumento syscall_id en C
 push r4, lr @ Preservamos lr para poder retornar correctamente
bl kernel_service @ Se llama a la funcion C del kernel pop r4,
pc @ Restauramos y regresamos al modo usuario
```

La instrucción `mrs` (*Move from Special Register*) es necesaria porque el **MSP** es un registro especial del sistema y no se puede leer directamente como $r0$ o $r12$, solo el modo privilegiado puede acceder a ella, lo que refuerza la seguridad del sistema.

Paso 5. El Dispatcher valida y enruta (`kernel_service.c`)

La función `kernel_service()` recibe el apuntador al *stack frame* (*svc_ args*) y el ID de la syscall. Usando un switch determinamos qué operación ejecutar y la validamos antes de ejecutar:

```c
    void kernel_service(uint32_t *svc_args, uint32_t syscall_id) {
      switch (syscall_id) {
        case SYS_GPIO_SET:
        // Set the GPIO value using the kernel function
        k_gpio_set(svc_args[0], svc_args[1]);
        break;

        case SYS_GPIO_GET:
        //Read the pin state via the hardware driver and write the result back into svc_args[0]
        //Because svc_args points to r0 on the user stack,
        //   this is equivalent to returning the value directly into r0
        svc_args[0] = (uint32_t)k_gpio_get(svc_args[0]);
        break;

        case SYS_GPIO_DIR:
        // Validate before executing.
        // Example: Prohibit touching pins 0 and 1 (UART) or pins > 28 (System)
        int pin = svc_args[0];
        if (pin < 2 || pin > 28) {
```

```
            svc_args[0] = -1; // Error: Permission denied (EACCES)
          } else {
            //Pin is within the safe range
            //Initialize direction via the driver and return 0 as signal success back to user space through r0
            k_gpio_init(svc_args[0], svc_args[1]);
            svc_args[0] = 0; //success
          }
          break;

          default:
          // Invalid syscall ID, return an error code (e.g., -1)
          svc_args[0] = -1;
          break;
        }
      }

```

La línea `svc_args[0] = resultado` es la forma en que el kernel retorna un valor al usuario. Como `svc_args[0]` apunta exactamente a donde está $r0$ guardado en el stack, al modificarlo el procesador restaurará ese valor en $r0$ cuando regrese al modo usuario.

Paso 6. El Driver manipula el hardware (`kernel_drivers.c`)

Este archivo es el único archivo del sistema con acceso directo a los registros físicos del RP2040 en `0xD0000000`. Usa apuntadores volátiles para acceder a los registros mapeados en memoria:

```
    void k_gpio_set(uint32_t pin, uint32_t value)
      {
        //Writing a bitmask of (1 << pin) to SET drives the pin high,
        //  writing the same mask to CLR drives it low
        //This avoids race conditions if an interrupt fires between the read and the write
        if (value)
        *(volatile uint32_t *)SIO_GPIO_OUT_SET = (1 << pin);
        else
        *(volatile uint32_t *)SIO_GPIO_OUT_CLR = (1 << pin);
      }

      int k_gpio_get(uint32_t pin)
      {
        //SIO_GPIO_IN holds the live state of all 30 GPIO pins simultaneously, one bit per pin
        //A volatile pointer forces the compiler to re-read the register
        //  on every access instead of caching a stale value
```

```c
        volatile uint32_t *GPIO_INPUT_STATE = (volatile uint32_t *)SIO_GPIO_IN;

        //Shift the register right by 'pin' positions to move the target bit
        //  into bit 0, then mask with 1 to discard all other pins
        //Ex: pin = 2, register = 0b...100 -> >> 2 -> 0b...1 -> & 1 -> 1
        return (int)((*GPIO_INPUT_STATE >> pin) & 1);
      }

```

Paso 7. El Retorno al Modo Usuario.

Una vez que `kernel_service()` termina, el control regresa a `wrapper_svc` (con pop $\{r4, pc\}$), y el procesador ARM restaura automáticamente desde el stack los registros $r0$, $r1$, $r2$, $r3$, $r12$, LR, PC y $xPSR$. La ejecución continúa exactamente en la instrucción siguiente al `svc #0` en `syscalls.S`:

```
svc #0 @ aqui entramos al kernel mov r7, r12 @ continuamos al
volver, r0 ya tiene el resultado bx lr @ regresamos a user_app.c
```

Aplicación de Usuario

Fase A: Puente Directo Botón → LED

En la Fase A el objetivo es simple, el LED simplemente refleja el estado del botón en tiempo real. No hay memoria de estado; el LED solo sigue al botón. Se usa `BTN_STATE` porque el LED funciona como un espejo instantáneo del botón.

Fase B

Sin embargo, a Fase B implementa un comportamiento más sofisticado: el LED actúa como un interruptor de memoria. Cada pulsación cambia su estado, independientemente de cuánto tiempo se mantenga presionado el botón, por ello gaurdamos `LED_STATE`, donde indicamos si debe estar encendido o apagado y esa memoria solo cambia en el momento exacto del flanco.

- **Detección de Flanco de Bajada.** Un flanco de bajada (*falling edge*) es el instante exacto en que una señal pasa de nivel alto (1) a nivel bajo (0). Para el botón con pull-up, esto ocurre justo cuando se presiona. Detectar el flanco en lugar del nivel permite que el toggle ocurra una sola vez por pulsación.

- **Software Debounce.** Los botones mecánicos tienen contactos metálicos que literalmente rebotan al juntarse, produciendo múltiples transiciones 0 y 1 en los

primeros 5 o 20 milisegundos. Sin debounce, cada presión física podría detectarse múltiples flancos.

Códigos completos

```c
#include <stdint.h>
#include "syscalls.h"

/* GPIO LED pin definition */
#define LED_PIN 13
/* GPIO Button pin definition */
#define BTN_PIN 2

/* Auxiliary function that performs a busy-wait delay  */
void
sys_delay(uint32_t time) {
  for (volatile uint32_t i = 0; i <= time; i ++);
}

int user_app_entry(void) {
  // Setup Phase (Unprivileged): The user program configures its resources using syscalls.

  //Configure LED_PIN as output (1) and BTN_PIN as input (0)
  //Neither call touches hardware directly, both go through the
  //  svc mechanism so the kernel can validate and execute them
  sys_gpio_dir(LED_PIN, 1);
  sys_gpio_dir(BTN_PIN, 0);
  int LED_STATE = 0; //track the current state of the LED
  int BTN_STATE = 0; //hold the current button reading from the syscall

  //hold the previous button state
  //Initialized to 1, because the pull-up resistor holds the pin high at rest
  int P_BTN_STATE = 1;

  while (1) {
    // Operational Phase (Unprivileged): The user program performs its main functionality.

    //Read the current button state via syscall.
    //Kernel reads the SIO_GPIO_IN register and returns the corresponding bit
    BTN_STATE = sys_gpio_get(BTN_PIN);

    //Falling edge detection:
    //  the button was released (1) last iteration and is now pressed (0) this iteration
    //This fires exactly once per physical press, not continuously while held down
    if (P_BTN_STATE == 1 && BTN_STATE == 0) {

      //Software debounce: wait and re-read the pin
```

```c
      //Mechanical buttons bounce between 0 and 1 for a few milliseconds after being pressed
      //If the pin is still 0/1 after the delay, the press is genuine and not electrical noise
      sys_delay(500000);
      if (sys_gpio_get(BTN_PIN) == 0) {

        //Toggle the LED state and apply it via syscall
        //Due to FASE B: LED_STATE is used instead of BTN_STATE because
        //  the LED must remember its own state independently of the button position
        LED_STATE = !LED_STATE;
    sys_gpio_set(LED_PIN, LED_STATE);
      }
    }

    //Store the current reading as the previous state for the next iteration
    //This is what makes falling edge detection possible on the next cycle
    P_BTN_STATE = BTN_STATE;
  }

  // Should never reach here
  return 0;
}
```

Código 1: user_app.c

```c
/*
 * @brief Kernel service handler for system calls.
 * @param svc_args: Pointer to the user stack frame.
 * @param syscall_id: The value of r7 passed by the assembler
 */
void kernel_service(uint32_t *svc_args, uint32_t syscall_id) {
  switch (syscall_id) {
  case SYS_GPIO_SET:
    // Set the GPIO value using the kernel function
    k_gpio_set(svc_args[0], svc_args[1]);
    break;

  case SYS_GPIO_GET:

    //Read the pin state via the hardware driver and write the result back into svc_args[0]
    //Because svc_args points to r0 on the user stack,
    //   this is equivalent to returning the value directly into r0
    svc_args[0] = (uint32_t)k_gpio_get(svc_args[0]);
    break;

  case SYS_GPIO_DIR:
    // Validate before executing.
    // Example: Prohibit touching pins 0 and 1 (UART) or pins > 28 (System)
    int pin = svc_args[0];
    if (pin < 2 || pin > 28) {
      svc_args[0] = -1; // Error: Permission denied (EACCES)
    } else {
      //Pin is within the safe range
      //Initialize direction via the driver and return 0 as signal success back to user space through r0
      k_gpio_init(svc_args[0], svc_args[1]);
      svc_args[0] = 0; //success
    }
    break;

  default:
    // Invalid syscall ID, return an error code (e.g., -1)
    svc_args[0] = -1;
    break;
  }
}
```

Código 2: kernel_service.c

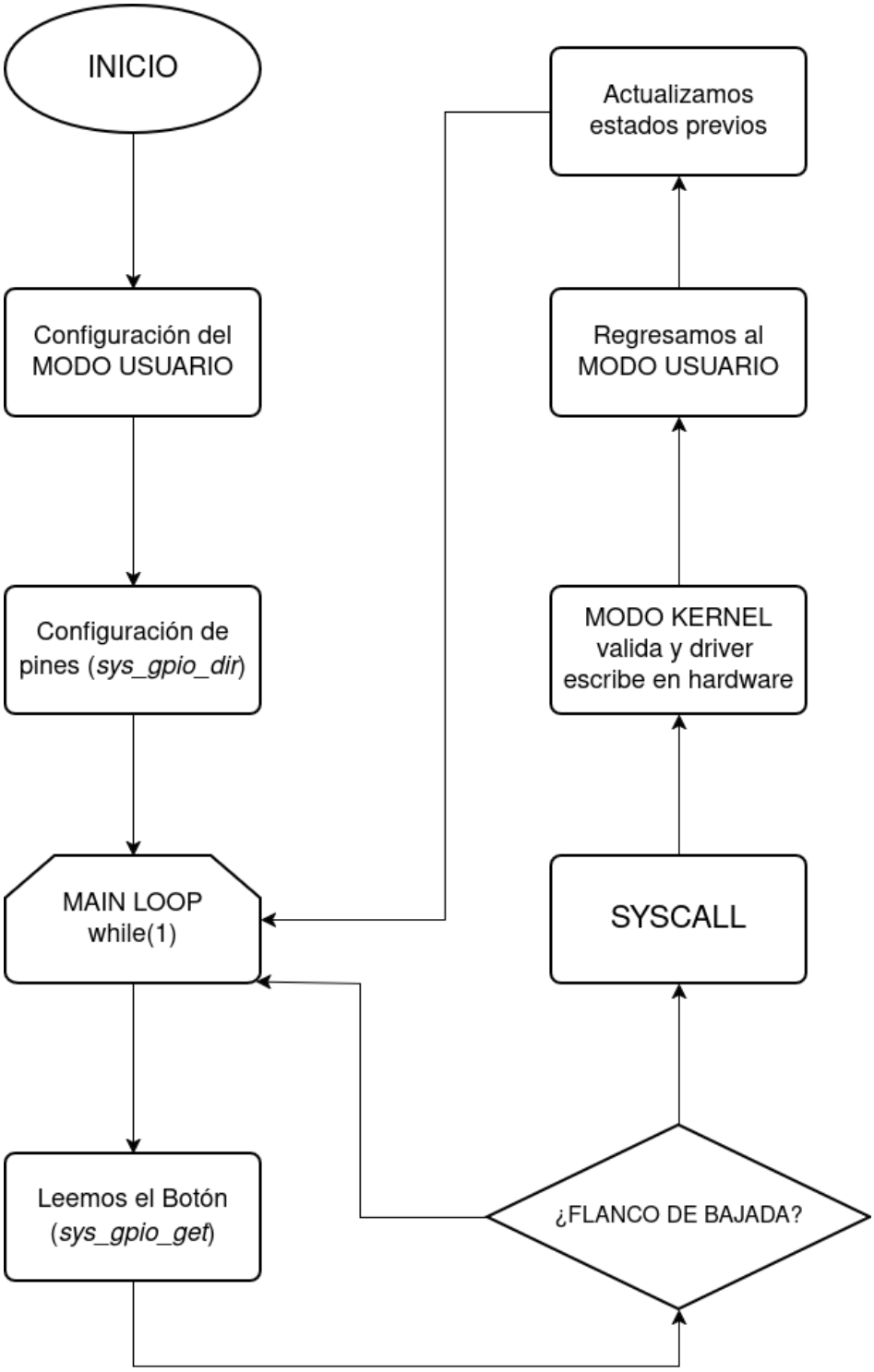
```c
//Pad control registers set electrical properties of each GPIO pin
//  such as drive strength, slew rate, and pull-up/pull-down resistors

//Base address of the PADS_BANK0 peripheral on the RP2040.
#define PADS_BANK0_BASE 0x4001C000

//Computes the address of the pad control register for a given pin
#define PADS_BANK0_GPIO(pin) (PADS_BANK0_BASE + 0x004 + (pin) * 4) //this

//Pull-Up Enable, setting this bit connects an internal resistor between the pin
//  and 3.3V, ensuring the pin reads 1 (high) when nothing is driving it
#define PADS_PUE (1 << 3)

//Pull-Down Enable
#define PADS_PDE (1 << 2)

/**
 * @brief Initialize GPIO pin as input or output and set function to SIO (func 5)
 * @param pin GPIO pin number
 * @param output 1 for output, 0 for input
 */
void k_gpio_init(uint32_t pin, uint32_t output)
{
  /* Select SIO function (func 5) for the pin */
  volatile uint32_t *gpio_ctrl = (volatile uint32_t *)IO_BANK0_GPIO_CTRL(pin);
  *gpio_ctrl = (*gpio_ctrl & ~0x1F) | GPIO_FUNC_SIO;

  /* Configure direction (output/input) */
  if (output)
    *(volatile uint32_t *)SIO_GPIO_OE_SET = (1 << pin);
  else {
    *(volatile uint32_t *)SIO_GPIO_OE_CLR = (1 << pin);

    //Get a pointer to this pin's pad control register
    volatile uint32_t *pad = (volatile uint32_t *)PADS_BANK0_GPIO(pin);

    //Enable the internal pull-up and explicitly disable the pull-down (PDE=0)
    //Both bits must be set correctly, because activating pull-up alone while pull-down remains
    //  enabled causes them to conflict, leaving the pin in an indeterminate floating state
    *pad = (*pad | PADS_PUE) & ~PADS_PDE;
  }
}
```

```c

/**
 * @brief Set GPIO pin output value
 * @param pin GPIO pin number
 * @param value 1 for high, 0 for low
 */
void k_gpio_set(uint32_t pin, uint32_t value)
{
  //Writing a bitmask of (1 << pin) to SET drives the pin high,
  //  writing the same mask to CLR drives it low
  //This avoids race conditions if an interrupt fires between the read and the write
  if (value)
    *(volatile uint32_t *)SIO_GPIO_OUT_SET = (1 << pin);
  else
    *(volatile uint32_t *)SIO_GPIO_OUT_CLR = (1 << pin);
}

/**
 * @brief Get GPIO pin input value
 * @param pin GPIO pin number
 * @return 1 if high, 0 if low
 */
int k_gpio_get(uint32_t pin)
{
  //SIO_GPIO_IN holds the live state of all 30 GPIO pins simultaneously, one bit per pin
  //A volatile pointer forces the compiler to re-read the register
  //  on every access instead of caching a stale value
  volatile uint32_t *GPIO_INPUT_STATE = (volatile uint32_t *)SIO_GPIO_IN;

  //Shift the register right by 'pin' positions to move the target bit
  //  into bit 0, then mask with 1 to discard all other pins
  //Ex: pin = 2, register = 0b...100 -> >> 2 -> 0b...1 -> & 1 -> 1
  return (int)((*GPIO_INPUT_STATE >> pin) & 1);
}
```

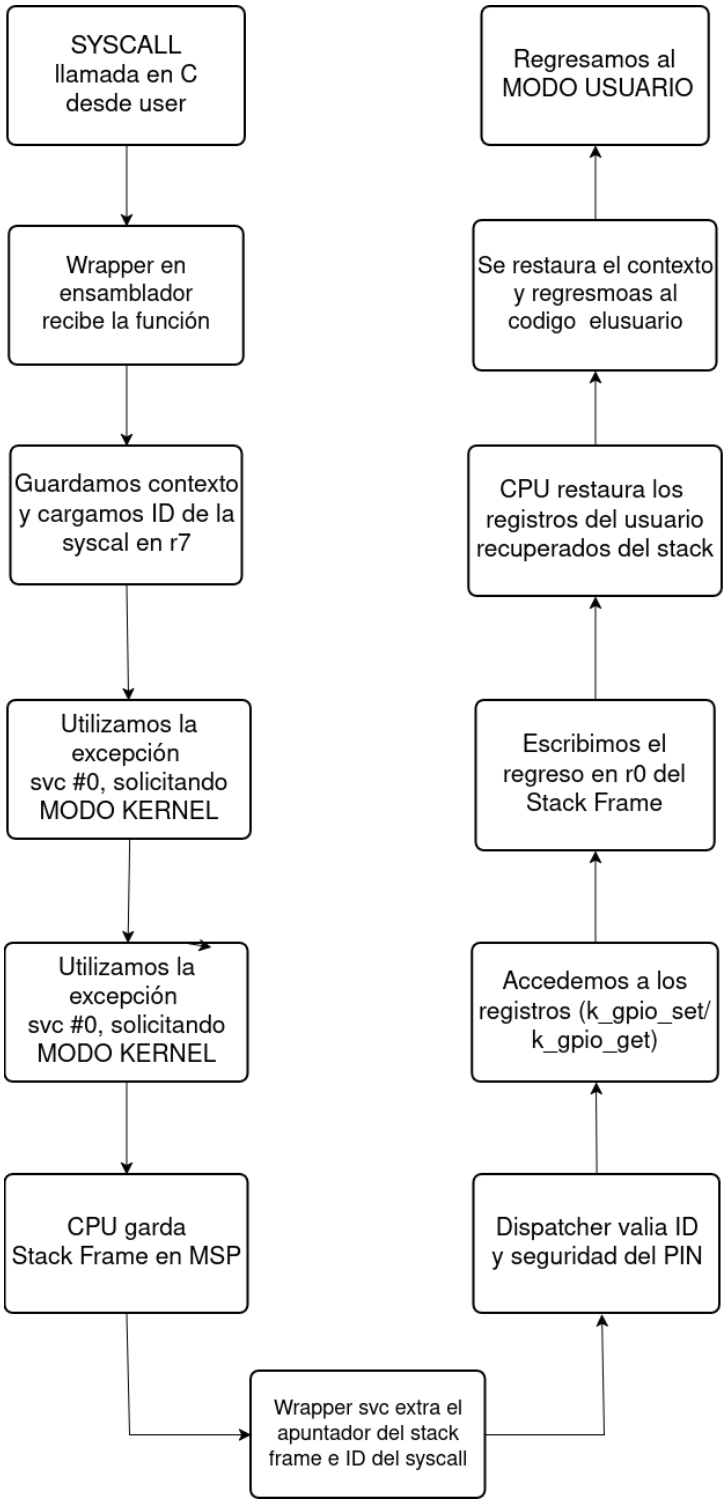
Código 3: kernel_drivers.c

Diagrama de flujo del programa

Tenemos un diagrama general que da la idea principal de como funciona la práctica:



Además tenemos el diagrama más detallado del flujo de la llamada al sistema y como lo maneja el modo kernel:



Evidencias

Evidencia Práctica 2 Video

Concluciones

En esta práctica mostramos con código y la placa Raspberry PiPico un diseño muy primitivo pero eficiente en la seguridad de sistemas operativos modernos:

- **Aislamiento de Privilegios:** La aplicación de usuario nunca accede directamente a `0xD0000000` ni a ningún registro de hardware. La barrera es absoluta y está reforzada por la arquitectura ARM: el modo usuario simplemente no puede ejecutar las instrucciones `mrs MSP` ni acceder a los registros de control del procesador.

- **Diseño de una ABI funcional:** El código de usuario la usa de forma transparente: llama funciones C que internamente tienen ensamblador invisible.

- **Gestión de Hardware Segura:** El kernel rechaza activamente peticiones inválidas. El dispatcher centralizado garantiza que ningún driver se ejecute sin pasar por validación.

- **Detección de Flanco vs. Nivel:** La condición P = 1, C = 0 implementa un detector de flanco de bajada en software con un bajo costo computacional.

- **Debounce por Confirmación:** El patrón de esperar y re-confirmar es más robusto que simplemente esperar un retardo fijo, porque elimina tanto el ruido del flanco inicial como el riesgo de que un retardo insuficiente no cubra todos los rebotes.

- **El Bug del Pull-Up/Pull-Down:** La necesidad de desactivar `PADS_PDE` al activar `PADS_PUE` es un ejemplo de cómo el hardware tiene estados por defecto que pueden no ser los que se espera al programarlos. Por lo que leer el datasheet y entender el valor inicial de cada registro es claramente indispensable en programación de sistemas embebidos.

  El sistema implementado es, en cierto modo (de manera muy reducida), equivalente al mecanismo de syscalls de Linux en arquitectura **ARM**: misma instrucción (`svc`), mismo registro de ID ($r7$), mismo mecanismo de paso de argumentos ($r0$-$r3$), mismo método de retorno ($r0$); pero en una escala menor, solo para enceder/apagar un LED mediante un botón.